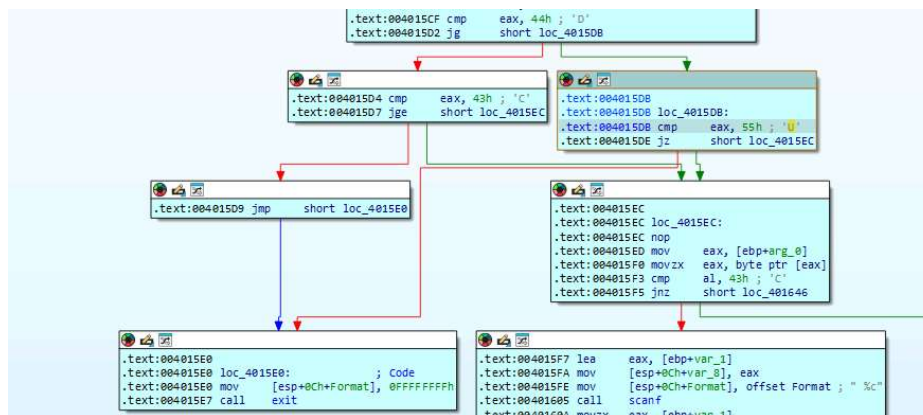


We had one last safe to analyze. Like the previous ones, it contained only a single file: safe.exe. As always, our goal was to reverse-engineer it and determine the correct input.

We began as usual by searching for user input handling, specifically looking for scanf calls. Surprisingly, the program had only two calls to scanf, both within the same function (40159C).

```
.text:00401580      sub_40159C      call  scanf
.text:00401605      sub_40159C      call  scanf
```

From static analysis, we learned that the function first reads a character, which must be either C, U, or D—any other input causes the program to exit.



If the character is C, an additional numeric character is read.

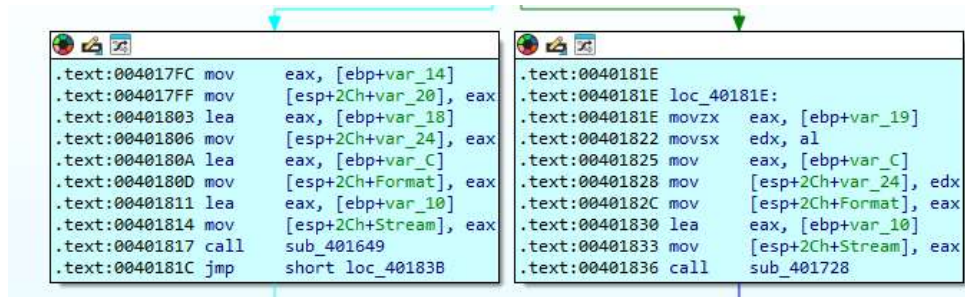
```
.text:004015F7 lea    eax, [ebp+var_1]
.text:004015FA mov     [esp+0Ch+var_8], eax
.text:004015FE mov     [esp+0Ch+Format], offset Format ; "%c"
.text:00401605 call    scanf
.text:0040160A movzx   eax, [ebp+var_1]
.text:0040160E movsx   eax, al
.text:00401611 mov     [esp+0Ch+Format], eax
.text:00401614 call    sub_401410
.text:00401619 movzx   eax, [ebp+var_1]
.text:0040161D cmp     al, 2Fh ; '/'
.text:0040161F jle     short loc_401629
```

At the end of the input process, the function stores the command and its associated number (in numeric form) in the variables it received via pointers.

At this point, the allowed inputs were still ambiguous, so we investigated the function

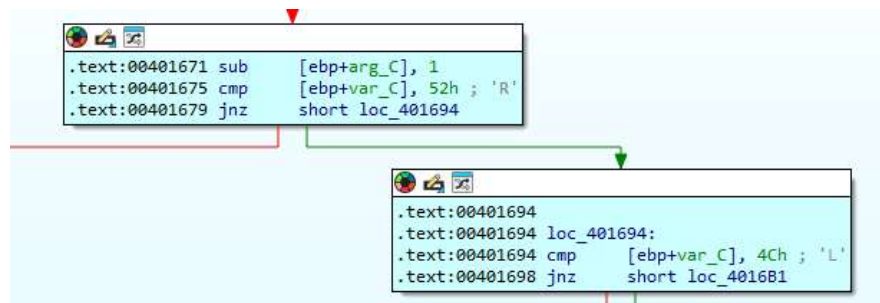
that calls the input routine (*sub_4017A7*). We'll refer to this as the main routine. After returning from the input routine, the program takes one of two paths:

- One for the case of C with a number (we'll call this the left routine),
- Another for U or D (the right routine).



The left routine receives the input number and three variables by reference from the main function. The right routine receives the character and two of the variables. This suggests that the left routine modifies all three variables, while the right one changes only the last.

At the beginning of the left routine, there is a check to see if the third variable holds the value L or R. Since U and D are already present, we infer these letters indicate directions: Up, Down, Left, Right. The third variable holds the horizontal direction.



At the start of the right routine, the program accesses a specific memory region, repeatedly referenced throughout the code. The memory access follows this formula:
 $base + var_1 * 8 + var_2$

```

.text:0040172E mov     eax, [ebp+arg_8]
.text:00401731 mov     [ebp+var_C], al
.text:00401734 mov     eax, [ebp+arg_0]
.text:00401737 mov     eax, [eax]
.text:00401739 mov     [ebp+var_4], eax
.text:0040173C mov     eax, [ebp+var_4]
.text:0040173F lea     edx, ds:0[eax*8]
.text:00401746 mov     eax, [ebp+arg_4]
.text:00401749 add     eax, edx
.text:0040174B add     eax, offset unk_408020
.text:00401750 movzx   eax, byte ptr [eax]
.text:00401753 mov     [ebp+var_5], al
.text:00401756 cmp     [ebp+var_5], 2Eh ; '.'

```

This resembles access to an 8-column 2D array starting at base. We tested this hypothesis and found that elsewhere in the code, a condition checks whether the two indices are within 0–7, confirming that this is indeed an 8x8 array:

- $var_1 = \text{row index}$
- $var_2 = \text{column index}$

We noted that the base array is dynamically initialized before the first input read. So we ran the program in debug mode, observed the array after initialization, and visualized it as an 8x8 matrix:

```

00408020  2E 2E 07 46 2E 04 2E 2E 58 58 58 58 58 58 58 58 ...F....XXXXXXXX
00408030  2E 2E 2E 2E 2E 2E 2E 2E 58 58 58 58 58 58 58 58 .....XXXXXXXX
00408040  46 2E 2E 2E 2E 2E 2E 2E 02 2E 2E 2E 2E 2E 2E 2E F.....
00408050  58 58 58 58 58 58 58 58 2E 03 2E 2E 2E 2E 2E 2E XXXXXXXX.....
00408060  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....

```

Figure 1: mem. table

.	.	7	F	.	4	.	.
X	X	X	X	X	X	X	X
.	.	.	.	.	.	.	.
X	X	X	X	X	X	X	X
F	.	.	.	.	.	.	.
2	.	.	.	.	.	.	.
X	X	X	X	X	X	X	X

.	3	.	.	.	.	.	.
---	---	---	---	---	---	---	---

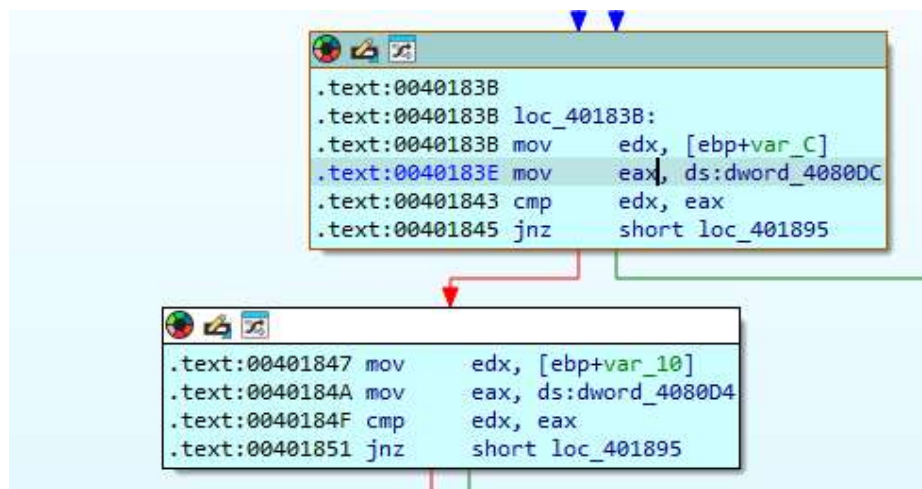
Table 1: reconstructed table

This clearly resembles a map, and based on previous findings, we realized the program simulates a game with directional movement.

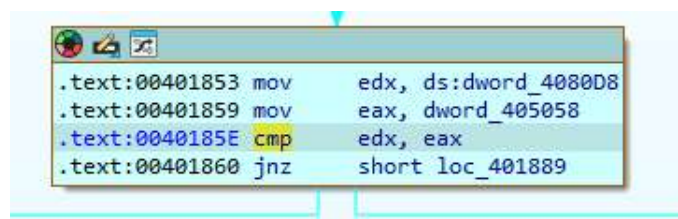
From code inspection, we saw the game starts with: (0,7).

That is, the starting position is at the top-right corner.

At the end of the main routine, a check determines if x and y have reached (7,7)—indicating the bottom-right corner. So, the goal is to reach the lower-right corner.



There is also a check for a global variable equal to 2.



We found this value increases only when the player reaches a cell containing F in the left routine.

```

.text:004016F6 mov     eax, ds:dword_4080D8
.text:004016FB add     eax, 1
.text:004016FE mov     ds:dword_4080D8, eax

```

When this happens, the F is removed and a score counter is incremented. This means the player must collect two Fs along the way.

Game Rules:

Left Routine (C command):

The direction (L or R) is stored in a variable.

$C(k)$ performs the move k times (recursion):

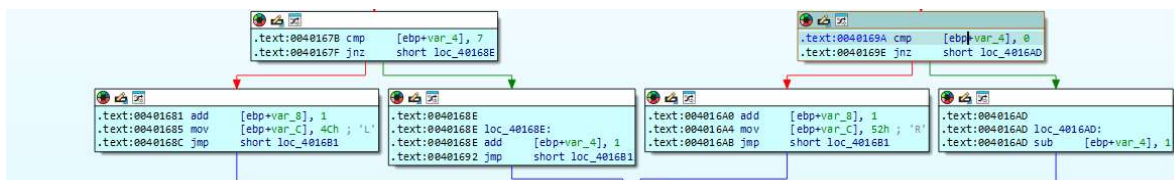
If direction is L, decrement x .

If direction is R, increment x .

If x exceeds bounds (0–7), x is not changed, but:

y is incremented (i.e., go down a row),

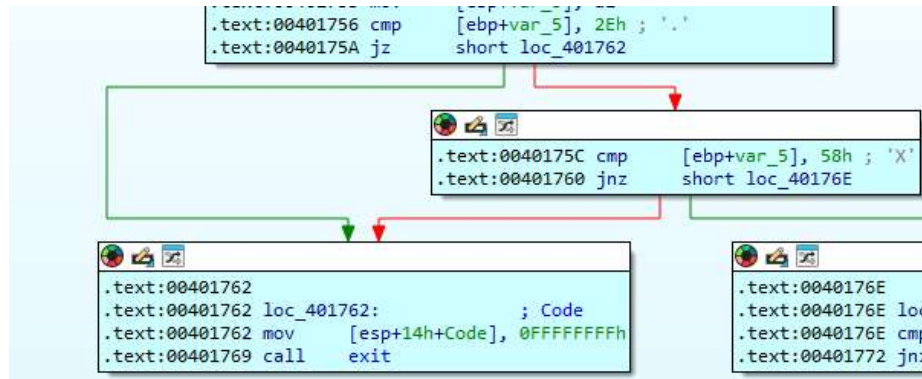
Direction is reversed.



This means C lets us move horizontally with wrapping to the next row if we hit the edge.

Right Routine (U/D command):

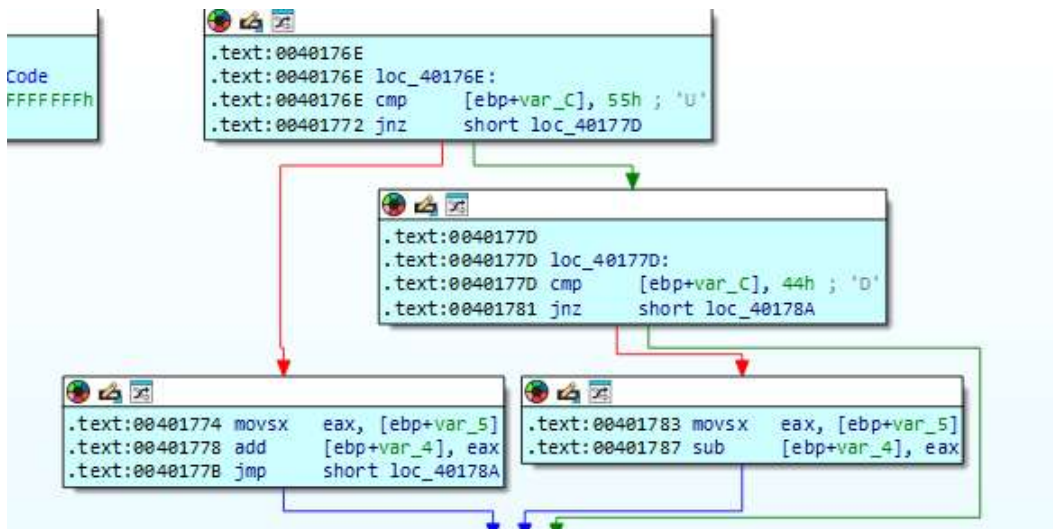
Allowed only on number or F cells.



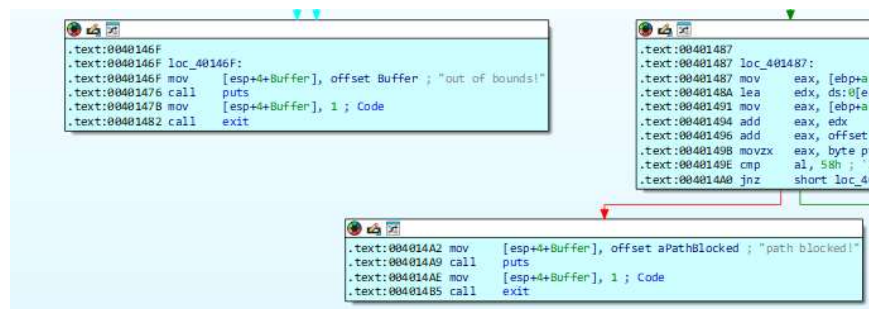
If current cell is a number and:

Command is U, the number is added to y.

Command is D, the number is subtracted from y.



If resulting cell is out of bounds or an X, the move is invalid.



Strategy and Solution:

Now that we understand the mechanics, we can determine the optimal path:

Start: top-right ($x = 0, y = 7$)

Goal: bottom-right ($x = 7, y = 7$)

Collect all Fs (two in total).

Using C commands for horizontal movement and U/D commands for vertical jumps (based on cell numbers), we map out the path.

In the visual diagram:

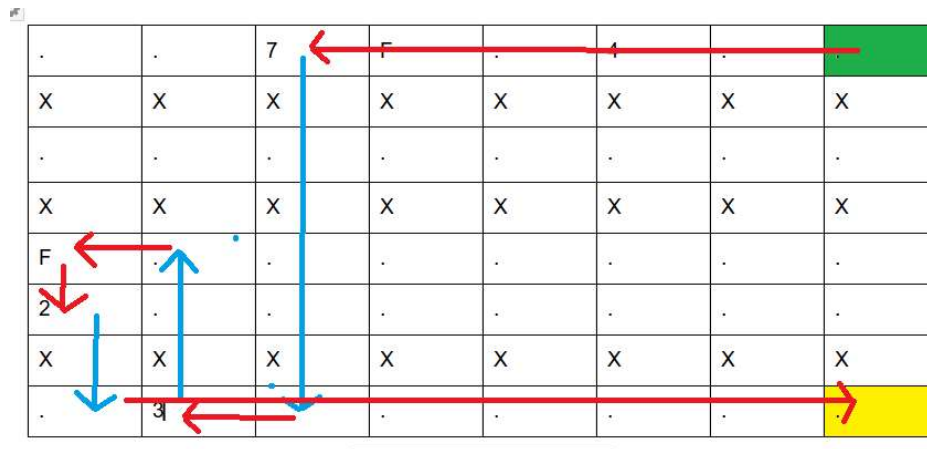
Green = Start

Yellow = End

Red arrows = horizontal C commands

Blue arrows = vertical U/D commands

We encountered a wall in row 4, so we changed direction accordingly.



The input sequence that wins the game is: `C 5 U C 1 D C 2 U C 7`.

```
C 5 U C 1 D C 2 U C 7
The encryption super secret key is 6!%b\+l2c%C,9VK]
```

The key is `6!%b\+l2c%C,9VK]`.